

OBJECT ORIENTED PROGRAMMING (CDT2001)

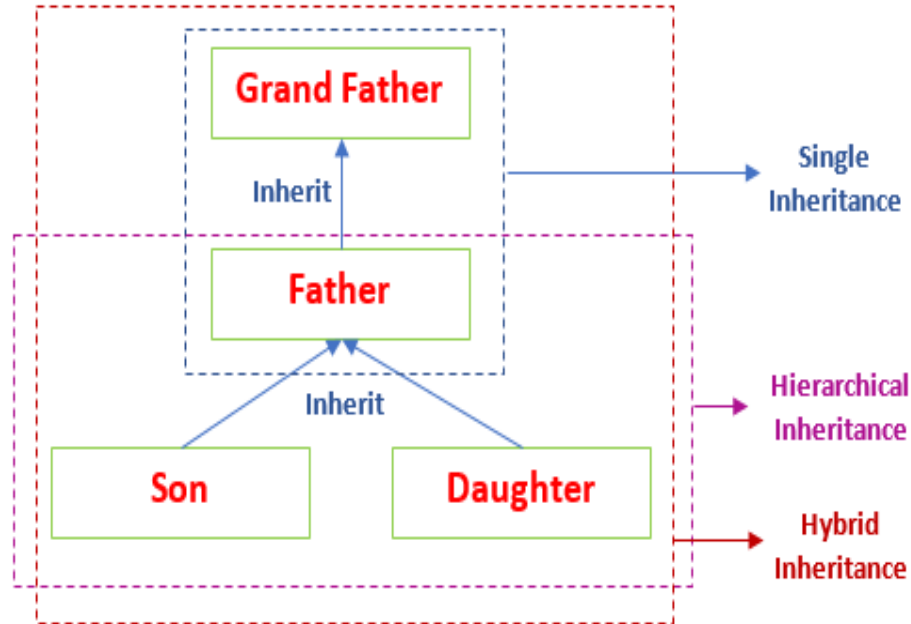
UNIT - III

- Concept of inheritance, methods of derivation, use of super keyword and final keyword in inheritance, run time polymorphism, abstract classes and methods , Interface, implementation of interface, static and non-static members.

- Concept Of Inheritance
- Use Of Super Keyword And Final Keyword In Inheritance
- Run Time Polymorphism
- Abstract Classes And Methods
- Interface And Implementation Of Interface
- Creating Packages And Importing Packages
- Static And Non-static Members
- Lambda Expressions

- Inheritance is the process by which one object acquires all the properties and behaviours of another object.
- It supports the concept of hierarchical classification.
- It is the mechanism in java by which one class is allows inheriting the features (fields and methods) of another class.
- The idea behind inheritance in Java is that you can create new classes that are built upon existing classes.
- When you inherit from an existing class, you can reuse methods and fields of the parent class.
- Moreover, you can add new methods and fields in your current class also.
- Inheritance represents the **IS-A relationship** which is also known as a parent-child relationship.

Example



Why use inheritance in java

- **Code Reusability:** The code written in the Superclass is common to all subclasses. Child classes can directly use the parent class code.
- **Method Overriding:** Method Overriding is achievable only through Inheritance. It is one of the ways by which Java achieves Run Time Polymorphism.
- **Abstraction:** The concept of abstract where we do not have to provide all details is achieved through inheritance. Abstraction only shows the functionality to the user.

Terms used in Inheritance

- **Class:** A class is a group of objects which have common properties. It is a template or blueprint from which objects are created.
- **Sub Class/Child Class:** Subclass is a class which inherits the other class. It is also called a derived class, extended class, or child class.
- **Super Class/Parent Class:** Superclass is the class from where a subclass inherits the features. It is also called a base class or a parent class.
- **Reusability:** As the name specifies, reusability is a mechanism which facilitates you to reuse the fields and methods of the existing class when you create a new class. You can use the same fields and methods already defined in the previous class.

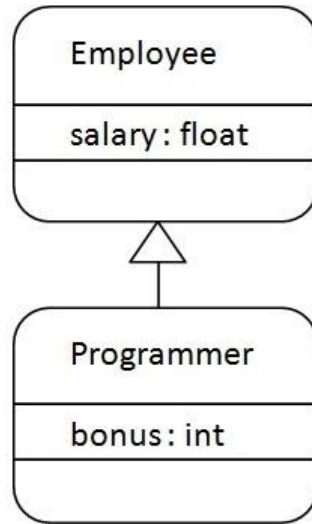
Syntax of Java Inheritance

```
class Subclass-name extends Superclass-name  
{  
    //methods and fields  
}
```

- The ***extends*** keyword indicates that you are making a new class that derives from an existing class
- The meaning of "***extends***" is to increase the functionality
- In the terminology of Java, a class which is **inherited** is called a **parent or superclass**, and the **new class** is called **child or subclass**

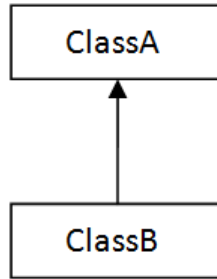
Inheritance Example

- Programmer is the subclass and Employee is the superclass.
- The relationship between the two classes is **Programmer IS-A Employee.**
- It means that Programmer is a type of Employee.

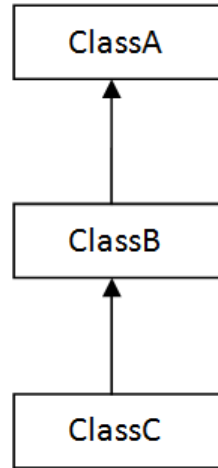


Types of inheritance in java

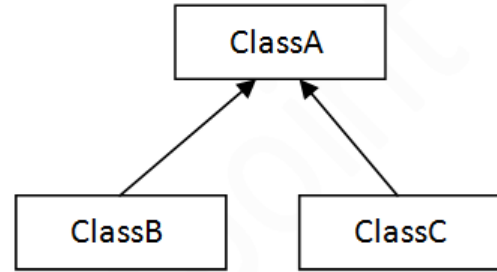
- On the basis of class, there can be three types of inheritance in java: single, multilevel and hierarchical



1) Single

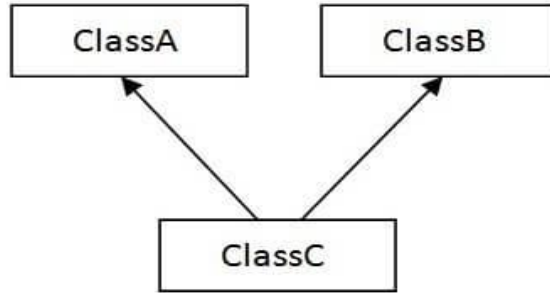


2) Multilevel

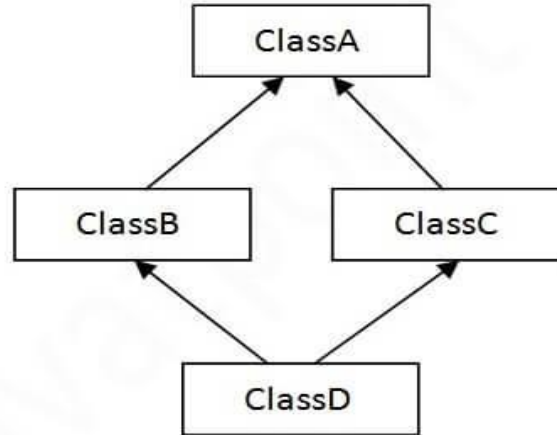


3) Hierarchical

Types of inheritance in java



4) Multiple

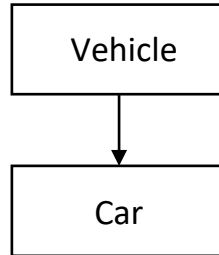


5) Hybrid

Note: Multiple inheritance is not supported in Java through class. It is achieved by interfaces.

Single Inheritance

- Refers to a child and parent class relationship where a class extends the another class
- When a class inherits another class, it is known as a single inheritance.
- In the example given below, Car class inherits the Vehicle class, so there is the single inheritance.



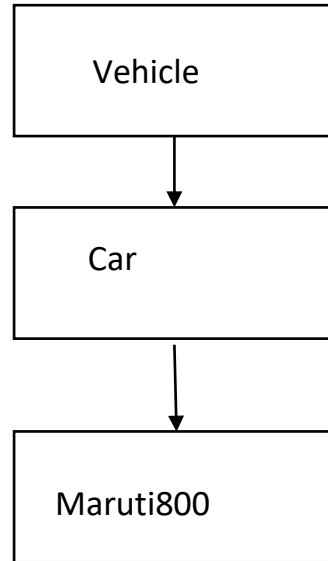
Sample Program- Single Inheritance

```
class Vehicle
{
    public void hasGear()
    {
        System.out.println("Has a gear");
    }
    public void hasBreak()
    {
        System.out.println("Has a Break");
    }
}
class Car extends Vehicle
{
    public void hasLowCost()
    {
        System.out.println("Has Low Cost");
    }
    public void hasHighSpeed()
    {
        System.out.println("Has High Speed");
    }
}

}

public class Inherit
{
    public static void main(String args[])
    {
        Car C1 = new Car();
        C1. hasLowCost();
        C1. hasHighSpeed();
        C1.hasGear();
        C1.hasBreak();
    }
}
```

- When there is a chain of inheritance, it is known as multilevel inheritance
- In the example, Maruti800 class inherits the Car class which again inherits the Vehicle class, so there is a multilevel inheritance



Sample Program- Multilevel Inheritance

```
class Vehicle
{
    public void hasGear()
    {
        System.out.println("Has a gear");
    }
    public void hasBreak()
    {
        System.out.println("Has a Break");
    }
}

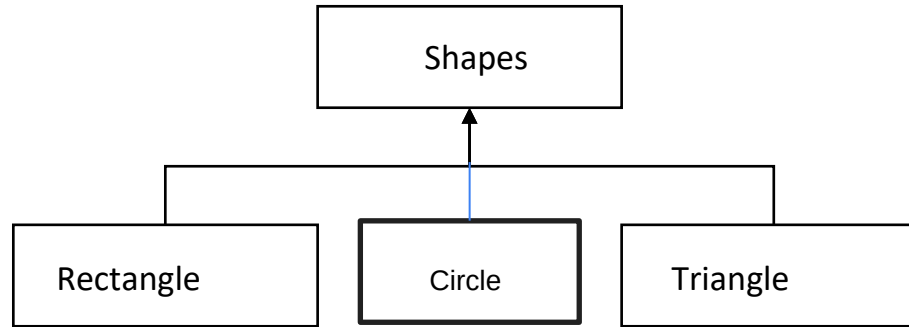
class Car extends Vehicle
{
    public void hasLowCost()
    {
        System.out.println("Has Low Cost");
    }
    public void hasHighSpeed()
    {
        System.out.println("Has High Speed");
    }
}
```

```
class Maruti800 extends Car
{
    public void hasAC()
    {
        System.out.println("Has AC");
    }
}

public class Inherit
{
    public static void main(String args[])
    {
        Maruti800 maruti = new Maruti800();
        maruti.hasLowCost();
        maruti.hasHighSpeed();
        maruti.hasGear();
        maruti.hasBreak();
        maruti.hasAC();
    }
}
```

Hierarchical Inheritance

- When two or more classes inherit a single class, it is known as hierarchical inheritance
- In the example, Rectangle, Triangle, and Circle classes inherit the Shapes class, so there is hierarchical inheritance.



Sample Program- Hierarchical Inheritance

```
class Shape {
    public void Shaped() {
        System.out.println("Base Class - Shape");
    }
}
class Rectangle extends Shape {
    public void Rectangled() {
        System.out.println("Derived Class -
Rectangle");
    }
}
class Triangle extends Shape {
    public void Triangled() {
        System.out.println("Derived Class - Triangle");
    }
}
class Circle extends Shape {
    public void Circled() {
        System.out.println("Derived Class - Circle");
    }
}
```

```
class JavaExample
{
    public static void main(String args[]) {
        Rectangle Rect = new Rectangle();
        Triangle Tri = new Triangle();
        Circle Cir = new Circle();
        //All classes can access the method of class
        Shape
        Rect.Shaped();
        Tri.Shaped();
        Cir.Shaped();
    }
}
```

- Java supports *single inheritance*, meaning that a derived class can have only one parent class
- *Multiple inheritance* allows a class to be derived from two or more classes, inheriting the members of all parents
- Collisions, such as the same variable name in two parents, have to be resolved
- **Java does not support multiple inheritance**
- In most cases, the use of **interfaces** gives us aspects of multiple inheritance without the overhead

Try yourself

.Define a class Student with roll number, name and total and method member show() to display roll number and name. Derive a class ScienceStudent from Student class which has physics , chemistry and math marks and showDetails() and findTotal() as method members. Using these classes create scienceStudent and display complete information about the student

Super Keyword

- The **super** keyword in java is a reference variable which is used to refer immediate parent class object
- Whenever you create the instance of subclass, an instance of parent class is created implicitly which is referred by super reference variable

Characteristics of super keyword:

- **super is used to call a superclass constructor:** When a subclass is created, its constructor must call the constructor of its parent class. This is done using the super() keyword, which calls the constructor of the parent class.
- **super is used to call a superclass method:** A subclass can call a method defined in its parent class using the super keyword. This is useful when the subclass wants to invoke the parent class's implementation of the method in addition to its own.
- **super is used to access a superclass field:** A subclass can access a field defined in its parent class using the super keyword. This is useful when the subclass wants to reference the parent class's version of a field.

Usage of Java super Keyword

- super can be used to refer immediate parent class instance variable
- super can be used to invoke immediate parent class method
- super() can be used to invoke immediate parent class constructor
- To access the data members of parent class when both parent and child class have member with same name
- To explicitly call the no-arg and parameterized constructor of parent class
- To access the method of parent class when child class has overridden that method.

How to use super keyword to access the variables of parent class

- When you have a variable in child class which is already present in the parent class then in order to access the variable of parent class, you need to use the super keyword.

Syntax: `super.variable_name`

Output:
110

Example - With Super Keyword

```
public class SuperSolutionExample {  
  
    int num = 100;  
}  
class Subclass extends SuperSolutionExample  
{  
    int num = 110;  
    void printNumber(){  
        /* Note that instead of writing num we are  
        * writing super.num in the print statement  
        * this refers to the num variable of Superclass  
        */  
        System.out.println(super.num);  
    }  
    public static void main(String args[]){  
        Subclass obj= new Subclass();  
        obj.printNumber();  
    }  
}
```

Output:
100

Use of super keyword to invoke constructor of parent class

- When we create the object of sub class, the new keyword invokes the constructor of child class, which implicitly invokes the constructor of parent class
- So in order to execution when we create the object of child class is: parent class constructor is executed first and then the child class constructor is executed
- It happens because compiler itself adds `super()`(this invokes the no-arg constructor of parent class) as the first statement in the constructor of child class

Example - Use of super keyword to invoke constructor of parent class

```
class SuperConstructorExample {
    SuperConstructorExample(){
        System.out.println("Constructor of parent class");
    }
}

class SubclassConstructor extends SuperConstructorExample {
    SubclassConstructor(){
        /* Compile implicitly adds super() here as the first
        statement of this constructor. */
        System.out.println("Constructor of child class");
    }
    SubclassConstructor(int num){
        /* Even though it is a parameterized constructor. it will invoke arg constructor of child class which will
        * The compiler still adds the no-arg super() here
        */
        System.out.println("arg constructor of child
class");
    }
    void display(){
        System.out.println("Hello!");
    }
}
```

```
class Demo {
    public static void main(String args[]){
        /* Creating object using default constructor. This
        will invoke child class constructor, which will
        * invoke parent class constructor */
        SubclassConstructor obj1= new
        SubclassConstructor();
        //Calling sub class method
        obj1.display();

        /* Creating second object using arg constructor */
        SubclassConstructor obj2= new
        SubclassConstructor(10);
        obj2.display();
    }
}
```

Parameterized super() call to invoke parameterized constructor of parent class

- When we have a constructor in parent class that takes arguments then we can use parameterized super, like `super(100);` to invoke parameterized constructor of parent class from the constructor of child class

There are few important points to note in this example:

- `super()`(or parameterized super) must be the first statement in constructor otherwise you will get the compilation error: “Constructor call must be the first statement in a constructor”
- When we explicitly placed super in the constructor, the java compiler didn't call the default no-arg constructor of parent class.



Example - Parameterized super() call to invoke parameterized constructor of parent class

```
class SuperExample2 {  
    //no-arg constructor  
    SuperExample2(){  
        System.out.println("no-arg constructor of parent  
class");  
    }  
    //arg or parameterized constructor  
    SuperExample2(String str){  
        System.out.println("parameterized constructor of  
parent class");  
    }  
}  
class SubclassExample2 extends SuperExample2  
{  
    SubclassExample2(){  
        /* super() must be added to the first statement of constructor  
otherwise you will get a compilation error. Another important  
        * point to note is that when we explicitly use super  
in constructor the compiler doesn't invoke the parent constructor  
automatically.*/  
        super("Hi");  
        System.out.println("Constructor of child class");  
    }  
}
```

```
void display(){  
    System.out.println("Hello");  
}  
}  
class Demo{  
    public static void main(String args[])  
    {  
        SubclassExample2 obj= new SubclassExample2();  
        obj.display();  
    }  
}
```

- The final keyword in java is used to restrict the user.
- The java final keyword can be used in many context.
- Final can be:
 - variable
 - method
- The final keyword is a non-access modifier used for classes, attributes and methods, which makes them non-changeable (impossible to inherit or override).
- The final keyword is useful when you want a variable to always store the same value, like PI (3.14159...).

- If you make any variable as final, you cannot change the value of final variable(It will be constant)
- There is a final variable speedlimit, we are going to change the value of this variable, but It can't be changed because final variable once assigned a value can never be changed

Java Final Keyword

- ⇒ Stop Value Change
- ⇒ Stop Method Overriding
- ⇒ Stop Inheritance

Example- Java final variable

```
class Bike9
{
    final int speedlimit=90;//final variable
    void run()
    {
        speedlimit=400;
    }
    public static void main(String args[])
    {
        Bike9 obj=new Bike9();
        obj.run();
    }
}
```

Output:Compile Time Error

- If you make any method as final, you cannot override it.

```
class Bike
{
    final void run()
    {
        System.out.println("running");
    }
}
class Honda extends Bike
{
    void run()
    {
        System.out.println("running safely with 100kmph");
    }
    public static void main(String args[])
    {
        Honda honda= new Honda();
        honda.run();
    }
}
```

Output:Compile Time Error

- If you make any class as final, you cannot extend it.

```
final class Bike{}  
class Honda1 extends Bike  
{  
    void run()  
    {  
        System.out.println("running safely with 100kmph");  
    }  
    public static void main(String args[])  
    {  
        Honda1 honda= new Honda1();  
        honda.run();  
    }  
}
```

Output:Compile Time Error

- Polymorphism in Java is a concept by which we can perform a single action in different ways.
- Polymorphism is derived from 2 Greek words: poly and morphs. The word "poly" means many and "morphs" means forms.
- So polymorphism means many forms.

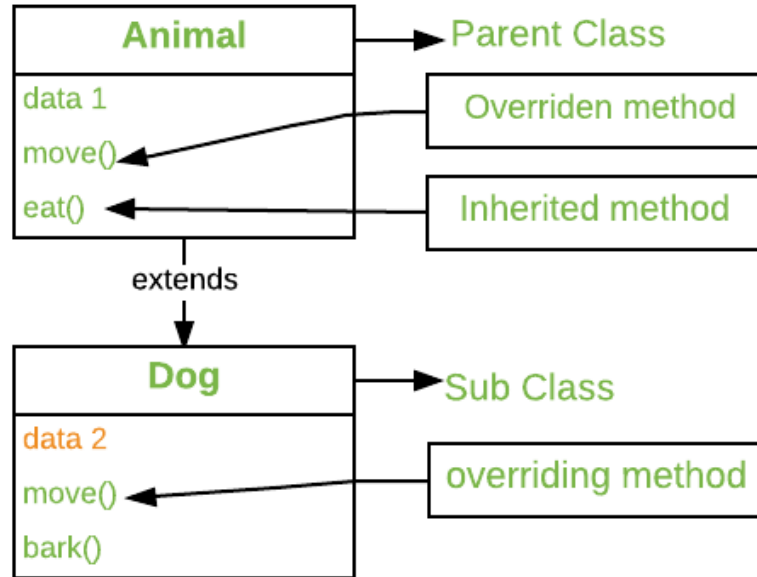
Types of Polymorphism

- **Static Polymorphism** also known as **compile time polymorphism**
 - Method overloading is an example of compile time polymorphism
- **Dynamic Polymorphism** also known as **runtime polymorphism**
 - Dynamic polymorphism is a process in which a call to an overridden method is resolved at runtime, that's why it is called runtime polymorphism

Method Overriding

- Declaring a method in sub class which is already present in parent class is known as method overriding.
- Overriding is done so that a child class can give its own implementation to a method which is already provided by the parent class.
- In this case the method in parent class is called overridden method and the method in child class is called overriding method.
- Method overriding is used to provide the specific implementation of a method which is already provided by its superclass.
- Method overriding is used for **runtime polymorphism**.

Method Overriding



Rules for Java Method Overriding

- The method must have the same name as in the parent class.
- The argument list and return type should be exactly the same as that of the overridden method.
- The access level cannot be more restrictive than the overridden method's access level. For example: If the superclass method is declared public then the overriding method in the sub class cannot be either private or protected.
- Instance methods can be overridden only if they are inherited by the subclass.
- A method declared final cannot be overridden.
- A method declared static cannot be overridden but can be re-declared.
- Constructors cannot be overridden.

Example 1- Method overriding

```
class Animal
{
    public void move()
    {
        System.out.println("Animals can move");
    }
}
class Dog extends Animal
{
    public void move()
    {
        System.out.println("Dogs can walk and run");
    }
}
```

```
public class TestClass
{
    public static void main(String args[])
    {
        Animal a = new Animal(); // Animal reference
        and object
        Animal d = new Dog(); // Animal reference but
        Dog object
        a.move(); // runs the method in Animal class
        d.move(); // runs the method in Dog class
    }
}
```

Output:-
Animals can move
Dogs can walk and run

Example 1- Method overriding

- In the above example, you can see that even though **b** is a type of Animal it runs the move method in the Dog class.
- The reason for this is: In compile time, the check is made on the reference type. However, in the runtime, JVM figures out the object type and would run the method that belongs to that particular object.
- Therefore, in the above example, the program will compile properly since Animal class has the method move. Then, at the runtime, it runs the method specific for that object.

Example 2 - Method overriding

```
public class Human
{
    //class Human
    //Overridden method
    public void eat()
    {
        System.out.println("Human is eating");
    }
}

class Boy extends Human {
    //Overriding method
    public void eat()
    {
        System.out.println("Boy is eating");
    }
}

public static void main( String args[]) {
    Boy obj = new Boy();
    //This will call the child class version of eat()
    obj.eat();
}
```

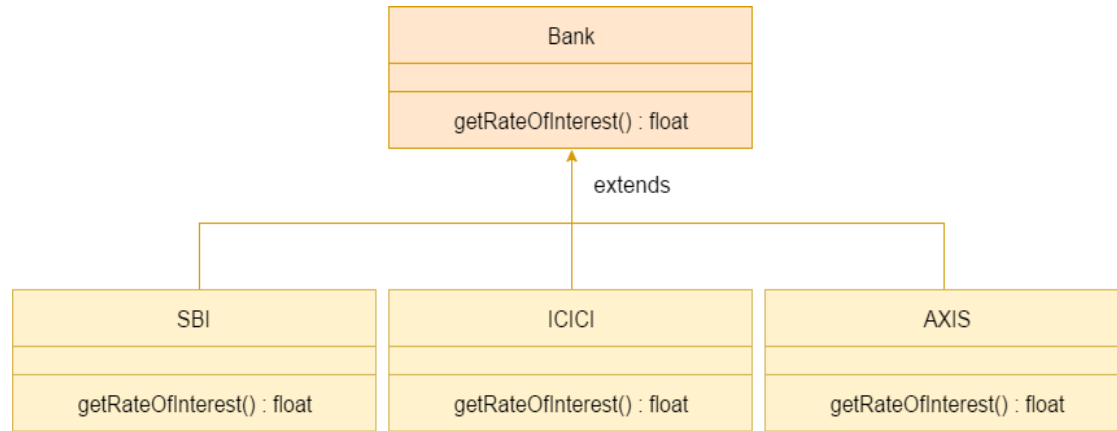
Example 2 - Method overriding

- We have two classes: A child class Boy and a parent class Human
- The Boy class extends Human class
- Both the classes have a common method void eat()
- Boy class is giving its own implementation to the eat() method or in other words it is overriding the eat() method
- Child class wants to give its own implementation so that when it calls this method, it prints Boy is eating instead of Human is eating

- The main advantage of method overriding is that the class can give its own specific implementation to a inherited method **without even modifying the parent class code.**
- This is helpful when a class has several child classes, so if a child class needs to use the parent class method, it can use it and the other classes that want to have different implementation can use overriding feature to make changes without touching the parent class code.

A real example of Java Method Overriding

- Consider a scenario where Bank is a class that provides functionality to get the rate of interest.
- However, the rate of interest varies according to banks.
- For example, SBI, ICICI and AXIS banks could provide 8%, 7%, and 9% rate of interest.



Method Overriding and Dynamic Method Dispatch

- Method Overriding is an example of **runtime polymorphism**
- When a parent class reference points to the child class object then the call to the overridden method is determined at runtime, because during method call which method(parent class or child class) is to be executed is determined by the type of object
- This process in which call to the overridden method is resolved at runtime is known as dynamic method dispatch
- When an overridden method is called through a superclass reference, Java determines which version(superclass/subclasses) of that method is to be executed based upon the type of the object being referred to at the time the call occurs. Thus, this determination is made at run time.
- At run-time, it depends on the type of the object being referred to (not the type of the reference variable) that determines which version of an overridden method will be executed
- A superclass reference variable can refer to a subclass object. This is also known as upcasting. Java uses this fact to resolve calls to overridden methods at run time.

Example - Dynamic Method Dispatch

```
public class MethodOverridingExample4 {
    //Overridden method
    public void disp()
    {
        System.out.println("disp() method of parent class");
    }
}

class Demo extends MethodOverridingExample4{
    //Overriding method
    public void disp()
    {
        System.out.println("disp() method of Child class");
    }
    public void newMethod()
    {
        System.out.println("new method of child class");
    }
    public static void main( String args[]) {
        /* When Parent class reference refers to the parent class object then in
        this case overridden method (the method of parent class is called. */
        MethodOverridingExample4 obj = new MethodOverridingExample4();
        obj.disp();
    }
}
```

```
/* When parent class reference refers to the child class object
then the overriding method (method of child class) is called. This is
called dynamic method dispatch and runtime polymorphism */
MethodOverridingExample4 obj2 = new Demo();
obj2.disp();
}
}
```

Super keyword in Method Overriding

- The super keyword is used for calling the parent class method/constructor.
- `super.myMethod()` calls the `myMethod()` method of base class while `super()` calls the constructor of base class

Super keyword in Method Overriding

```
public class MethodOverridingExample5 {
    public void myMethod()
    {
        System.out.println("Overridden method");
    }
}
class ABC extends MethodOverridingExample5{
    public void myMethod(){
        //This will call the myMethod() of parent class
        super.myMethod();
        System.out.println("Overriding method");
    }
    public static void main( String args[]) {
        ABC obj = new ABC();
        obj.myMethod();
    }
}
```


Difference between Method Overloading and Method Overriding

Method overloading	Method overriding
1. More than one method with same name, different prototype in same scope is called method overloading.	1. More than one method with same name, same prototype in different scope is called method overriding.
2. In case of method overloading, parameter must be different.	2. In case of method overriding, parameter must be same.
3. Method overloading is the example of compile time polymorphism.	3. Method overriding is the example of run time polymorphism.
4. Method overloading is performed within class.	4. Method overriding occurs in two classes.
5. In case of method overloading, Return type can be same or different.	5. In case of method overriding Return type must be same.
6. Static methods can be overloaded which means a class can have more than one static method of same name.	6. Static methods cannot be overridden, even if you declare a same static method in child class it has nothing to do with the same method of parent class.
7. <u>Static binding</u> is being used for overloaded methods	7. <u>dynamic binding</u> is being used for overridden/overriding methods.

The @Override Annotation in Java

- @Override annotation denotes that the child class method overrides the base class method.
- For two reasons, the @Override annotation is useful
 - If the annotated method does not actually override anything, the compiler issues a warning
 - It can help to make the source code more readable

Why we use @Override annotation:

- While overriding a method in the child class, we should use this annotation to mark that method
- This makes code readable and avoid maintenance issues, such as: while changing the method signature of parent class, you must change the signature in child classes (where this annotation is being used) otherwise compiler would throw compilation error.
- This is difficult to trace when you haven't used this annotation

@Override example

- In the example we are overriding a method displayMethod() in the child class
- Even if we don't use the @Override annotation, the program would still run fine without any issues

```
public class OverrideAnnotation
{
    public void displayMethod(String msg)
    {
        System.out.println(msg);
    }
}

class SubClass extends OverrideAnnotation
{
    @Override
    public void displayMethod(String msg)
    {
        System.out.println("Message is: "+ msg);
    }
}
```

```
public static void main(String args[])
{
    SubClass obj = new SubClass();

    obj.displayMethod("Hey!!");
}
```

Advantage @Override

- If programmer makes any mistake such as wrong method name, wrong parameter types while overriding, you would get a compile time error. As by using this annotation you instruct compiler that you are overriding this method. If you don't use the annotation then the sub class method would behave as a new method (not the overriding method) in sub class.
- It improves the readability of the code. So if you change the signature of overridden method then all the sub classes that overrides the particular method would throw a compilation error, which would eventually help you to change the signature in the sub classes. If you have lots of classes in your application then this annotation would really help you to identify the classes that require changes when you change the signature of a method.

Static Binding and Dynamic Binding

Association of method call to the method body is known as binding

- **Static Binding** that happens at **compile time**
- **Dynamic Binding** that happens at **runtime**

- The binding which can be resolved at compile time by compiler is known as static or early binding
- Consider two classes Human and Boy.
- Both the classes have same method walk() but the method is **static**, which means it cannot be overridden so even though I have used the object of Boy class while creating object obj, the parent class method is called by it, because the reference is of Human type (parent class).
- So whenever a binding of **static, private and final methods** happen, type of the class is determined by the compiler at compile time and the binding happens then and there.

Try yourself-Example

```
public class Human {

    public static void walk()
    {
        System.out.println("Human walks");
    }
}

class Boy extends Human
{
    public static void walk()
    {
        System.out.println("Boy walks");
    }
}
```

```
public static void main( String args[]) {
    /* Reference is of Human type and object is
       Boy type */
    Human obj = new Boy();
    /* Reference is of Human type and object is
       of Human type. */
    Human obj2 = new Human();
    obj.walk();
    obj2.walk();
}
```

- When compiler is not able to resolve the call/binding at compile time, such binding is known as **Dynamic or late Binding**
- **Method Overriding** is a perfect example of dynamic binding as in overriding both parent and child classes have same method and in this case the type of the object determines which method is to be executed
- The type of object is determined at the run time so this is known as dynamic binding
- Overriding is actually happening since these methods are **not static, private and final**
- In case of overriding the call to the overridden method is determined at runtime by the type of object thus late binding happens



Try yourself-Example

```
public class Human1
{
    //Overridden Method
    public void walk()
    {
        System.out.println("Human walks");
    }
}
class Demo extends Human1
{
    //Overriding Method
    public void walk()
    {
        System.out.println("Boy walks");
    }
}
```

```
public static void main( String args[])
{
    /* Reference is of Human type and object
    is Boy type */

    Human1 obj = new Demo();
    /* Reference is of Human type and object
    is of Human type. */

    Human1 obj2 = new Human1();
    obj.walk();
    obj2.walk();
}
```

Static vs Dynamic Binding

Static Binding	Dynamic Binding
It is a binding that happens at compile time.	It is a binding that happens at run time.
Actual object is not used for binding.	Actual object is used for binding.
It is also called early binding because binding happens during compilation.	It is also called late binding because binding happens at run time.
Method overloading is the best example of static binding.	Method overriding is the best example of dynamic binding.
Private, static and final methods show static binding. Because, they can not be overridden.	Other than private, static and final methods show dynamic binding. Because, they can be overridden.

Do it Yourself

A publishing company that markets both book and audiocassette versions of its works

Create a class publication that stores the title(a string) and price (float) of a publication

From this class derive two classes book which adds the page count (int) and tape which adds a playing time in minutes (int) each of these three classes should have a setData() function to initialise the data members and a display() function to display its data.

Write a main program to test the book and tape classes by creating instances of them

Write a program to demonstrate the concept of dynamic polymorphism for the publishing company.

Abstract Keyword

- Sometimes there are scenarios in which you will want to define super class that declares the structure of a given abstraction without providing a complete implementation of every method.
- So there is a need to define a generalized method which all subclasses will share, but doesn't need an implementation.
- Java's solution to this problem is the Abstract method
- Abstract means incomplete
- Java defines abstract method and abstract classes

- A class which is declared as abstract is known as an **abstract class**.
- It can have abstract and non-abstract methods.
- It needs to be **extended** and its method **implemented**.
- It **cannot be instantiated**.
- It can have final methods
- It can have constructors and static methods also
- It can have final methods which will force the subclass not to change the body of the method

Abstract class declaration

- An abstract class outlines the methods but not necessarily implements all the methods

/Declaration using abstract keyword

```
abstract class A{
```

```
    abstract void myMethod(); //This is abstract method
```

```
    //This is concrete method with body
```

```
    void anotherMethod(){
```

```
        //Does something
```

```
    }
```

```
}
```

Example of Abstract Class and Method -1

```
abstract class Figure {
    double dim1;
    double dim2;
    Figure(double a, double b) {
        dim1 = a;
        dim2 = b;
    }
    abstract double area();
}

class Rectangle extends Figure {
    Rectangle(double a, double b) {
        super(a, b);
    }
    double area() {
        System.out.println("Inside Area for Rectangle.");
        return dim1 * dim2;
    }
}

class Triangle extends Figure {
    Triangle(double a, double b) {
        super(a, b);
    }
}
```

```
double area() {
    System.out.println("Inside Area for Triangle.");
    return dim1 * dim2 / 2;
}

class AbstractAreas {
    public static void main(String args[]) {
        // Figure f = new Figure(10, 10); // illegal now
        Rectangle r = new Rectangle(9, 5);
        Triangle t = new Triangle(10, 8);
        Figure figref; // this is OK, no object is created
        figref = r;
        System.out.println("Area is " + figref.area());
        figref = t;
        System.out.println("Area is " + figref.area());
    }
}
```

Example of Abstract Class and Method

- In the given example, the program declares **area()** as abstract inside **Figure**. This means that all classes derived from **Figure** must override **area()**.
- As the comment inside **main()** indicates, it is no longer possible to declare objects of type **Figure**, since it is now abstract. And, all subclasses of **Figure** must override **area()**.
- To prove this to yourself, try creating a subclass that does not override **area()**. You will receive a compile-time error.
- Although it is not possible to create an object of type **Figure**, you can create a reference variable of type **Figure**.
- The variable **figref** is declared as a reference to **Figure**, which means that it can be used to refer to an object of any class derived from **Figure**.
- As explained, it is through superclass reference variables that overridden methods are resolved at run time.

- **Note 1:** As we seen in the above example, there are cases when it is difficult or often unnecessary to implement all the methods in parent class. In these cases, we can declare the parent class as abstract, which makes it a special class which is not complete on its own. A class derived from the abstract class must implement all those methods that are declared as abstract in the parent class.
- **Note 2:** Abstract class cannot be instantiated which means you cannot create the object of it. To use this class, you need to create another class that extends this class and provides the implementation of abstract methods, then you can use the object of that child class to call non-abstract methods of parent class as well as implemented methods(those that were abstract in parent but implemented in child class).
- **Note 3:** If a child does not implement all the abstract methods of abstract parent class, then the child class must need to be declared abstract as well.

Important Points

- An abstract class has no use until unless it is extended by some other class
- If you declare an abstract method in a class then you must declare the class abstract as well. you can't have abstract method in a concrete class. It's vice versa is not always true: If a class is not having any abstract method then also it can be marked as abstract
- It can have non-abstract method (concrete) as well.
- Data abstraction is the process of hiding certain details and showing only essential information to the user
- Abstraction can be achieved with either abstract classes or interfaces
- **Abstract class:** is a restricted class that cannot be used to create objects (to access it, it must be inherited from another class)
- **Abstract method:** can only be used in an abstract class, and it does not have a body. The body is provided by the subclass (inherited from)

Example of Abstract Class and Method-2

```
abstract class Bank {  
  
    abstract int getRateOfInterest();  
}  
  
class SBI extends Bank{  
    int getRateOfInterest(){return 7;}  
}  
  
class PNB extends Bank{  
    int getRateOfInterest(){return 8;}  
}
```

```
class TestBank{  
  
    public static void main(String args[]){  
  
        Bank b;  
  
        b=new SBI();  
  
        System.out.println("Rate of Interest is:  
        "+b.getRateOfInterest()+" %");  
  
        b=new PNB();  
  
        System.out.println("Rate of Interest is:  
        "+b.getRateOfInterest()+" %");  
  
    }  
  
}
```

Example of Abstract Class and Method-3

```
abstract class Animal {  
    abstract void makeSound();  
    public void eat() {  
        System.out.println("I can eat.");  
    }  
}  
class Dog1 extends Animal {  
    public void makeSound() {  
        System.out.println("Bark bark");  
    }  
}
```

```
class Main1 {  
    public static void main(String[] args) {  
  
        // create an object of Dog class  
        Dog1 d1 = new Dog1();  
        d1.makeSound();  
        d1.eat();  
    }  
}
```

Java access modifiers:

- **Private:** The access level of a private modifier is only within the class. It cannot be accessed from outside the class.
- **Default:** The access level of a default modifier is only within the package. It cannot be accessed from outside the package. If you do not specify any access level, it will be the default.
- **Protected:** The access level of a protected modifier is within the package and outside the package through child class. If you do not make the child class, it cannot be accessed from outside the package.
- **Public:** The access level of a public modifier is everywhere. It can be accessed from within the class, outside the class, within the package and outside the package.

Java Modifiers (Access Modifiers)

- For classes, you can use either public or default:

Modifier	Description
public	The class is accessible by any other class
<i>default</i>	The class is only accessible by classes in the same package. This is used when you don't specify a modifier.

Java Modifiers (Access Modifiers)

- For attributes, methods and constructors, you can use the one of the following: (Access Modifiers)

Modifier	Description
public	The code is accessible for all classes
private	The code is only accessible within the declared class
<i>default</i>	The code is only accessible in the same package. This is used when you don't specify a modifier.
protected	The code is accessible in the same package and subclasses .

Understanding Java Access Modifiers

Access Modifier	within class	within package	outside package by subclass only	outside package
Private	Y	N	N	N
Default	Y	Y	N	N
Protected	Y	Y	Y	N
Public	Y	Y	Y	Y

Private Access Modifier

- The scope of private modifier is limited to the class only
- Private Data members and methods are only accessible within the class
- Class and Interface cannot be declared as private
- If a class has private constructor then you cannot create the object of that class from outside of the class

Example - Private Access Modifier

```
public class PrivateAccessModifier {  
    private double num = 100;  
    private int square(int a){  
        return a*a;  
    }  
}  
  
public class Example{  
    public static void main(String args[]){  
        PrivateAccessModifier obj = new PrivateAccessModifier();  
        System.out.println(obj.num);  
        System.out.println(obj.square(10));  
    }  
}
```

NOTE- Above program will give errors like

num has private access in PrivateAccessModifier Class

square(int)has private access in PrivateAccessModifier Class

Example - Private Access Modifier

```
public class PrivateAccessModifier2 {  
    private String fname = "John";  
    private String lname = "Doe";  
    private String email = "john@doe.com";  
    private int age = 24;  
  
    public static void main(String[] args) {  
        PrivateAccessModifier2 myObj = new PrivateAccessModifier2();  
        System.out.println("Name: " + myObj.fname + " " + myObj.lname);  
        System.out.println("Email: " + myObj.email);  
        System.out.println("Age: " + myObj.age);  
    }  
}
```

NOTE- Above program will run successfully because all private members are accessed inside the same class.

- Using the keyword **interface**, you can fully abstract a class' interface from its implementation.
- That is, using **interface**, you can specify what a class must do, but not how it does it.
- Interfaces are syntactically similar to classes, but they lack instance variables, and their methods are declared without any body. In practice, this means that you can define interfaces that don't make assumptions about how they are implemented.
- Once it is defined, any number of classes can implement an **interface**. Also, one class can implement any number of interfaces.
- To implement an interface, a class must create the complete set of methods defined by the interface.
- However, each class is free to determine the details of its own implementation.

- It has static, constants and abstract methods
- Interface looks like a class but it is not a class , the class that implements interface must implement all the methods of that interface
- An interface can have methods and variables just like the class but the methods declared in interface are by default abstract (only method signature, no body)
- Also, the variables declared in an interface are public, static & final by default
- An interface is a completely "abstract class" that is used to group related methods with empty bodies

Need of Interface

Solve the problem of multiple inheritance

How to declare an interface?

An interface is defined much like a class. This is the general form of an interface:

access interface name

```
{  
    return-type method-name1(parameter-list);  
    return-type method-name2(parameter-list);  
    type final-varname1 = value;  
    type final-varname2 = value;  
    return-type method-nameN(parameter-list);  
    type final-varnameN = value;  
}
```

- To access the interface methods, the interface must be "**implemented**" (like inherited) by another class with the **implements** keyword (instead of extends)
- The body of the interface method is provided by the "implement" class.

Example

```
Interface Callback{  
    void callback (int param);  
}  
class Client implements Callback {  
public void callback(int p)  
    {  
        System.out.println("callback called with " + p);  
    }  
void nonIfaceMeth()  
{  
    System.out.println("Classes that implement interfaces  
        may also define other members, too.");  
}  
}
```

```
class TestIface {  
public static void main(String args[]) {+  
    Client c = new Client();  
    c.callback(42);  
}  
}
```

The output of this program is shown here:
callback called with 42

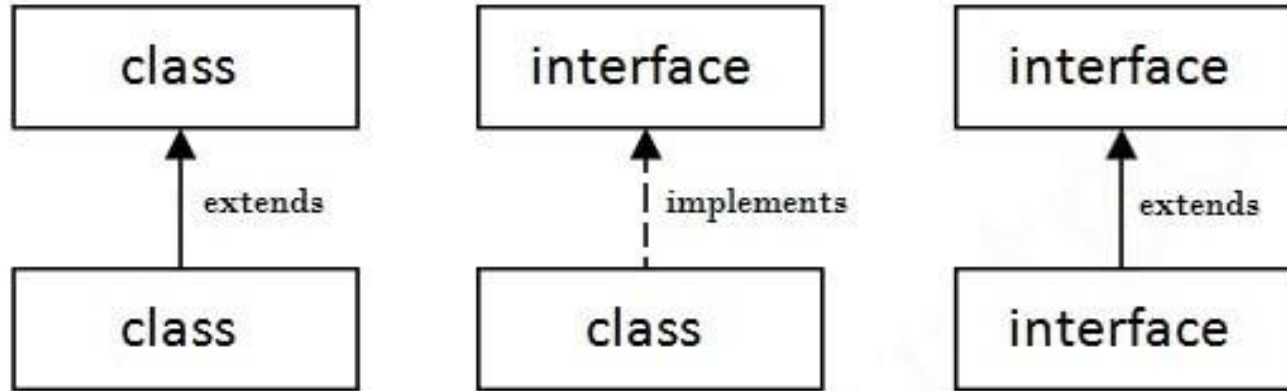
Example

```
Interface Callback{  
    void callback (int param);  
}  
class Client implements Callback {  
    public void callback(int p)  
    {  
        System.out.println("callback called with " + p);  
    }  
    void nonIfaceMeth()  
    {  
        System.out.println("Classes that implement interfaces  
            may also define other members, too.");  
    }  
}
```

```
class TestIface {  
    public static void main(String args[]) {  
        Callback c = new Client();  
        c.callback(42);  
    }  
}
```

The output of this program is shown here:
callback called with 42

The relationship between classes and interfaces



- An interface must be declared with the keyword interface.
- All variables defined in an interface must be public, static, and final.
- Interfaces can declare only constants, not instance variables.
- Interface methods cannot be marked final.
- An interface can extend one or more other interfaces.
- An interface cannot extend anything but another interface.
- An interface cannot implement another interface or class.
- Interface types can be used polymorphically.

Example 1- Implementing Single Interface

```
interface InterfaceExample1
{
    void print();
}
class A6 implements InterfaceExample1
{
    public void print()
    {
        System.out.println("Hello");
    }
    public static void main(String args[])
    {
        A6 obj = new A6();
        obj.print();
    }
}
```

Example 2- Implementing Multiple Interface

```
interface FirstInterface
{
    public void myMethod(); // interface method
}
interface SecondInterface
{
    public void myOtherMethod();
}
class DemoClass implements FirstInterface,
SecondInterface
{
    public void myMethod()
    {
        System.out.println("Some text from first interface..");
    }
    public void myOtherMethod()
    {
        System.out.println("Some text from second
```

```
interface...");
    }
}

class Main
{
    public static void main(String[] args)
    {
        DemoClass myObj = new DemoClass();
        myObj.myMethod();
        myObj.myOtherMethod();
    }
}
```

Example 3 - Implementing Hybrid Interface

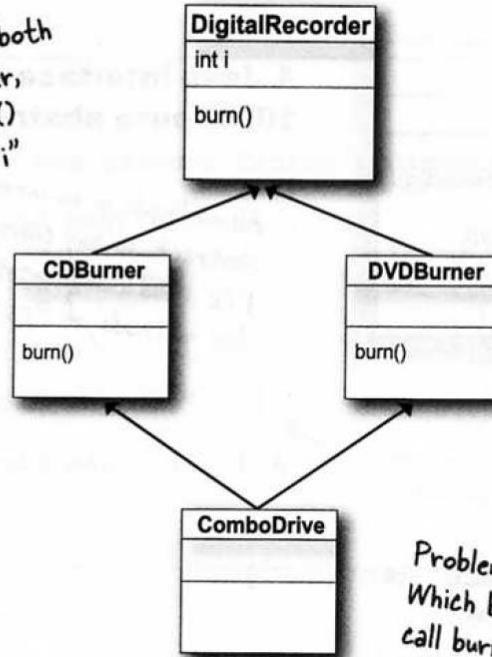
```
interface Shape
{
    void draw();
}
class Rectangle implements Shape
{
    public void draw()
    {
        System.out.println("drawing rectangle");
    }
}
class Circle implements Shape
{
    public void draw()
    {
        System.out.println("drawing circle");
    }
}
```

```
class TestInterface1
{
    public static void main(String args[])
    {
        Shape d=new Circle();
        d.draw();
    }
}
```

What is the problem with multiple inheritance??

Deadly Diamond of Death

CDBurner and DVDBurner both inherit from DigitalRecorder, and both override the burn() method. Both inherit the "i" instance variable.



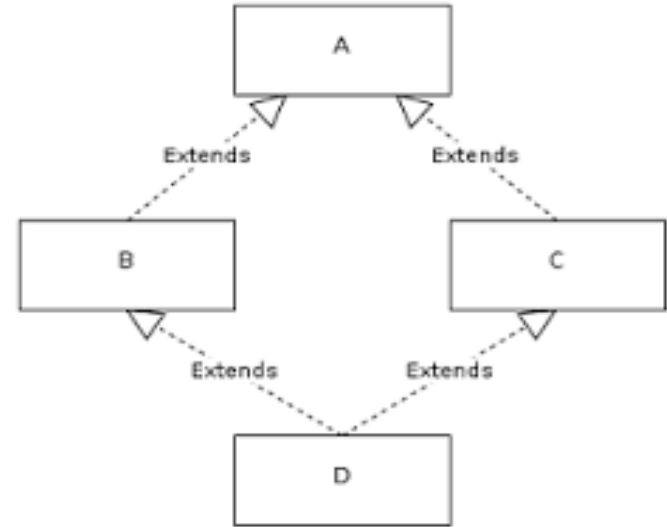
Imagine that the "i" instance variable is used by both CDBurner and DVDBurner, with different values. What happens if ComboDrive needs to use both values of "i"?

Problem with multiple inheritance. Which burn() method runs when you call burn() on the ComboDrive?

Multiple inheritance in Java by interface

Diamond Problem in Java

- Java does not allow multiple inheritance where one class can inherit properties from more than one class
- It is known as the diamond problem
- Class B and class C inherits the class A. The display() method of class A is overridden by the class B and class C
- Class D inherits the class B and class C which is invalid in Java
- Assume that we need to call the display() method by using the object of class D, in this scenario Java compiler does not know which display() method to call. Therefore, it creates ambiguity.



Solution to Diamond Problem

- The solution to the diamond problem is **default methods** and **interfaces**.
- We can achieve multiple inheritance by using these two things.
- The default method is similar to the abstract method.
- The only difference is that it is defined inside the interfaces with the default implementation.
- We need not to override these methods. Because they are already implementing these interfaces.
- The advantage of interfaces is that it can have the same default methods with the same name and signature in two different interfaces.
- It allows us to implement these two interfaces, from a class.
- We must override the default methods explicitly with its interface name.
- Syntax:

InterfaceName.**super**.methodName();

Solution to Diamond Problem-Example

```
interface DemoInterface1
{
    public default void display()
    {
        System.out.println("the display() method of
DemoInterface1 invoked");
    }
}
interface DemoInterface2
{
    public default void display()
    {
        System.out.println("the display() method of
DemoInterface2 invoked");
    }
}
public class MultipleInheritanceSolution1 implements
DemoInterface1, DemoInterface2
{
    @Override
    public void display()
    {
        DemoInterface1.super.display();
        DemoInterface2.super.display();
    }
    public static void main(String args[])
    {
        MultipleInheritanceSolution1 obj = new
MultipleInheritanceSolution1();
        obj.display();
    }
}
```

Key Points for Interface

- We can't instantiate an interface in java. That means we cannot create the object of an interface.
- Interface provides full abstraction as none of its methods have body. On the other hand abstract class provides partial abstraction as it can have abstract and concrete(methods with body) methods both.
- implements keyword is used by classes to implement an interface.
- While providing implementation in class of any method of an interface, it needs to be mentioned as public.
- Class that implements any interface must implement all the methods of that interface, else the class should be declared abstract.
- Interface cannot be declared as private, protected or transient.

Key Points for Interface

- All the interface methods are by default abstract and public
- Variables declared in interface are public, static and final by default

interface Try

{

int a=10;

public int a=10;

public static final int a=10;

final int a=10;

static int a=10;

}

- All of the above statements are identical

Key Points for Interface

- Interface variables must be initialized at the time of declaration otherwise compiler will throw an error

```
interface Try
```

```
{
```

```
    int x;//Compile-time error
```

```
}
```

- Above code will throw a compile time error as the value of the variable x is not initialized at the time of declaration

Key Points for Interface

- Inside any implementation class, you cannot change the variables declared in interface because by default, they are public, static and final.
- Implementation of the interface “Try” which has a variable x.
- The value for variable x is tried for setting but compilation error occurs as the variable x is public static final by default and final variables can not be re-initialized.

class Sample implements Try

```
{  
    public static void main(String args[])  
    {  
        x=20; //compile time error  
    }  
}
```

A class implements an interface, but one interface extends another interface.

```
interface Printable{
void print();
}
interface Showable extends Printable{
void show();
}
class TestInterface4 implements Showable{
public void print(){System.out.println("Hello");}
public void show(){System.out.println("Welcome");}
public static void main(String args[])
{ TestInterface4 obj = new TestInterface4();
  obj.print();
  obj.show();
}
```

- static Methods in Interface are those methods, which are defined in the interface with the keyword static.
- Unlike other methods in Interface, these static methods contain the complete definition of the function and since the definition is complete and the method is static, therefore these methods cannot be overridden or changed in the implementation class.
- Similar to Default Method in Interface, the static method in an interface can be defined in the interface, but cannot be overridden in Implementation Classes.
- To use a static method, Interface name should be instantiated with it, as it is a part of the Interface only.

Static Method in Interface

```
interface Drawable
{
void draw();
static int cube(int x)
{
return x*x*x;}
}
class Rectangle implements Drawable
{
public void draw()
{
System.out.println("drawing rectangle");
}
}
```

```
class TestInterfaceStatic
{
public static void main(String args[])
{
Drawable d=new Rectangle();
d.draw();
System.out.println(Drawable.cube(3));
}
}
```


- There are folders or directories in our computers for the classification and accessibility of various files, and in Java, we have packages for the same
- In Java, Packages are similar to folders, which are mainly used to organize classes and interfaces
- Java provides some built-in packages which we can use but we can also create our own (user-defined) packages
- Packages are used in Java in order to prevent naming conflicts, to control access, to make searching/locating and usage of classes, interfaces, enumerations and annotations easier, etc.
- *Packages* are containers for classes that are used to keep the class name space compartmentalized.
- For example, a package allows you to create a class named **List**, which you can store in your own package without concern that it will collide with some other class named **List** stored elsewhere.
- Packages are stored in a hierarchical manner and are explicitly imported into new class definitions.

Packages are used for

Packages in Java is a mechanism to encapsulate a group of classes, sub packages and Interfaces.

- **Preventing naming conflicts.** For example there can be two classes with name Teacher in two packages, college.staff.icse.Teacher and college.staff.cbse.Teacher
- **Making searching/locating** and usage of classes, interfaces, enumerations and annotations easier
- **Providing access control:** protected and default have package level access control. A protected member is accessible by classes in the same package and its subclasses. A default member (without any access specifier) is accessible by classes in the same package only
- Packages can be considered as **data encapsulation** (or data-hiding)
- **Reusability**
- **Better Organization**

Types of Packages

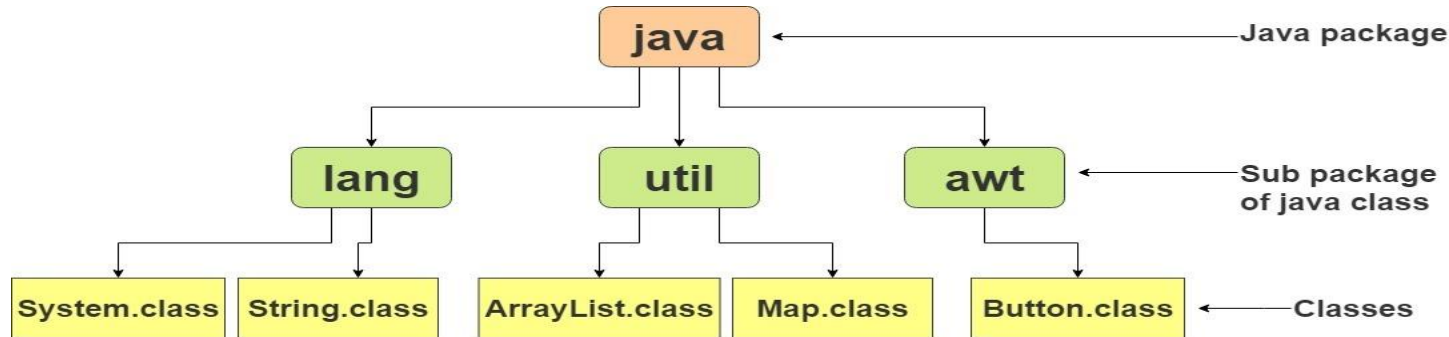
- **User defined package:** The package we create is called user-defined package.
- **Built-in package:** The already defined package like `java.io.*`, `java.lang.*` etc are known as built-in packages.
- for example when we need user input, we import a package like this:
- `import java.util.Scanner`

Here:

→ java is a top level package

→ util is a sub package

→ and Scanner is a class which is present in the sub package util.



1. `java.lang.*` : Defines actual language. Imported automatically. Includes classes for primitive types, strings, math function, threads and exceptions
2. `java.util.*` : Datastructures provided like hash tables, queues, etc.
3. `java.io.*` : Input/Output support classes for data I/O
4. `java.net.*` : Networking-classes for communicating with computers over the internet
5. `java.awt.*` : Windows in java. GUI classes for buttons, list, menus, etc.
6. `javax.swing.*` : GUI java's version of windows
7. `java.sql.*` : Database transactions
8. `java.net.http.*` : web programming
9. `java.applet.*` : Classes for creating and implementing applets

Defining a Package

- To create a package, include a package command as the first statement in a Java source file. Any classes declared within that file will belong to the specified package.
- The package statement defines a name space in which classes are stored. If you omit the package statement, the class names are put into the default package, which has no name.
- While the default package is fine for short, sample programs, it is inadequate for real applications. Most of the time, you will define a package for your code.
- This is the general form of the package statement:

```
package pkg;
```

Here, *pkg* is the name of the package.

- For example, the following statement creates a package called MyPackage.

```
package MyPackage;
```

Creating user defined packages

- Syntax

```
package<name of package>; //package declaration
class <class name>{
                                //class definition
                                //Body of class
}
```

- .java file must be saved in the directory having the name of specified package
- On compilation .class created in same directory

- Example

```
package mypack;

public class Simple{

    public static void main(String args[]){
        System.out.println("Welcome to package");
    }
}
```

Package firstpackage.secondpackage

- Allows similar classes to be grouped into same package
- Similar packages to be grouped into a larger packages

Ex. java.io.Writer

Accessing package

import package1.package2.....classname

Or

import packagename.*

Package Example

```
package package1;
```

```
public class MyDate{
```

```
    private int dd,mm,yy;
```

```
    public MyDate(){
```

```
        dd=1;mm=2;yy=2010;
```

```
    }
```

```
    public void showDate(){
```

```
        System.out.println(dd+"/"+mm+"/"+yy);
```

```
    }
```

```
}
```

```
import package1.MyDate;
```

```
class PackageDemo2{
```

```
    public static void main(String arg[]){
```

```
        MyDate d1 = new MyDate();
```

```
        MyDate d1 = new MyDate();
```

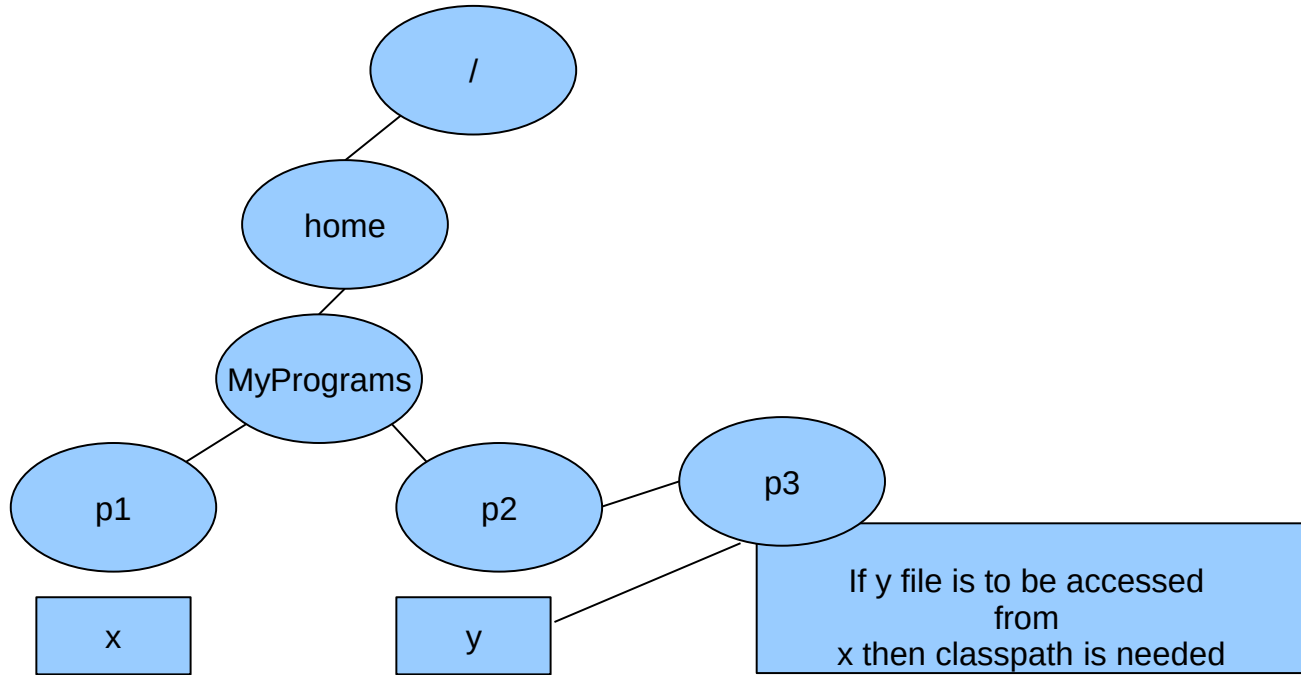
```
        d1.showDate();
```

```
        d2.showDate();
```

```
    }
```

```
}
```


Directory structure & classpath



Creating a class inside package while importing another package

The order in this case should be:

- → package declaration
- → package import

//Declaring a package

package anotherpackage;

//importing a package

import letmecalculate.Calculator;

public class Example{

 public static void main(String args[]){

 Calculator obj = new Calculator();

 System.out.println(obj.add(100, 200));

 }

}

Using fully qualified name while importing a class

- Calculator.java

```
package letmecalculate;
public class Calculator {
    public int add(int a, int b){
        return a+b;
    }
    public static void main(String args[]){
        Calculator obj = new
Calculator();
        System.out.println(obj.add(10,
20));
    }
}
```

- Example.java

```
//Declaring a package
package anotherpackage;
public class Example{
    public static void main(String args[]){
        //Using fully qualified name instead of
import
        letmecalculate.Calculator obj = new
        letmecalculate.Calculator();
        System.out.println(obj.add(100, 200));
    }
}
```

Subpackages:

- Packages that are inside another package are the subpackages
- These are not imported by default, they have to be imported explicitly
- If you import a package, all the classes and interfaces of that package will be imported excluding the classes and interfaces of the subpackages
- Hence, you need to import the subpackage as well
- Let's say I have created another package inside `calculate` and the subpackage name is `multiply`.
- So if I create a class in this subpackage it should have this package declaration in the beginning:
 - `package calculate.multiply;`

Default Access Modifier

- The scope of this modifier is limited to the package only
- This means that if we have a class with the default access modifier in a package, only those classes that are in this package can access this class
- No other class outside this package can access this class
- Similarly, if we have a default method or data member in a class, it would not be visible in the class of another package

Default Access Modifier Example

```
package abcpackage;
```

```
public class Addition {
```

```
    /* Since we didn't mention any access modifier  
    here, it would
```

```
    * be considered as default.
```

```
    */
```

```
    int addTwoNumbers(int a, int b){
```

```
        return a+b;
```

```
    }
```

```
}
```

OUTPUT: Exception in thread "main" java.lang.Error:
Unresolved compilation problem:
The method addTwoNumbers(int, int) from the type
Addition is not visible
at xyzpackage.Test.main(Test.java:12)

```
package xyzpackage;
```

```
/* We are importing the abcpackage
```

```
 * but still we will get error because the
```

```
 * class we are trying to use has default access
```

```
 * modifier.
```

```
*/
```

```
import abcpackage.*;
```

```
public class Test {
```

```
    public static void main(String args[]){
```

```
        Addition obj = new Addition();
```

```
    /* It will throw error because we are trying to access
```

```
    * the default method in another package
```

```
    */
```

```
        obj.addTwoNumbers(10, 21);
```

```
    }
```

```
}
```

Protected Access Modifier

- Protected data member and method are only accessible by the classes of the same package and the subclasses present in any package
- The protected access modifier is similar to default access modifier with one exception that it has visibility in sub classes
- Classes cannot be declared protected
- This access modifier is generally used in a parent child relationship

Protected Access Modifier - Example

- In this example the class Test which is present in another package is able to call the addTwoNumbers() method, which is declared protected
- This is because the Test class extends class Addition and the protected modifier allows the access of protected members in subclasses (in any packages)

```
package abcpackage;
```

```
public class addition {
```

```
    protected int addTwoNumbers(int a, int b)
    {
        return a+b;
    }
}
```

```
package xyzpackage;
import abcpackage.*;
```

```
class Test extends addition{
    public static void main(String args[]){
        Test obj = new Test();
        System.out.println(obj.addTwoNumbers(11, 22));
    }
}
```


Public Access Modifier

- The members, methods and classes that are declared public can be accessed from anywhere
- This modifier doesn't put any restriction on the access

```
package ABCpackage;
```

```
public class Addition {
```

```
    public int addTwoNumbers(int a, int b){  
        return a+b;  
    }  
}
```

```
package XYZpackage;  
import ABCpackage.*;
```

```
public class test {
```

```
    public static void main(String args[]){  
        Addition obj = new Addition();  
  
        System.out.println(obj.addTwoNumbers(100,  
1));  
    }  
}
```

'static' keyword

Static keyword can be used with

- Variable
- Method
- Block
- Nested class

Need of Static modifier

Consider a utility class to perform Math operations

```
class MathOp
```

```
{
```

```
    cube(int num){}
```

```
    square(int num){}
```

```
}
```

Here, cube and square methods always run the same way, no matter which instance of class called the method

- The method behavior does not depend on the state of an object
- Therefore, object is not required to call the method

Static variable

- **Belongs to class rather than any object of that class**
- **Can be accessed without having to create any instance of class**
- **Created only once** in memory
- Static variable is **1 per class** and instance variable is 1 per object
- It is **allocated memory in class area** when class is loaded for 1st time
- **All objects of a class share same copy** of static variable

Example

```
Class Frog
{
Static int frogCount=0;
Frog()
{
frogCount++;
}
Public static void main(String ar[])
{
Frog f1=new Frog();
Frog f2=new Frog();
new Frog();
SOP(frogCount);or SOP(Frog.frogCount);
}
```

Static method

- Belongs to class
- Can be invoked without creating any object of the class
- Static methods can access static data members and can change their values
- Static method is used to perform operations which do not belong to any object
- Static method cannot access non-static data members (instance variable) or invoke non-static methods directly

Example

```
class Frog{  
staticint frogCount=0;  
Frog(){  
frogCount++;  
}  
static int getCount(){  
return(frogCount);  
}
```

```
public static void main(String ar[]){  
Frog f1= new Frog();  
Frog f2=new Frog();  
new Frog();  
SOP(Frog.getCount());  
or  
SOP(getCount());  
or  
SOP(f1.getCount())  
}
```

- Instance Initializer block is used to initialize the instance data member
- the initializer block is used to declare/initialize the common part of various constructors of a class. It runs every time whenever the object is created.
- The initializer block contains the code that is always executed whenever an instance is created and it runs each time when an object of the class is created.
- There are 3 areas where we can use the initializer blocks:
 - Constructors
 - Methods
 - Blocks

Blocks in Java - Example

```
// Class
class Car {
    // Class member variable
    int speed;
    // Constructor
    Car()
    {
        // Print statement when constructor is called
        System.out.println("Speed of Car: " + speed);
    }
    // Block
    {
        speed = 60;
    }
    // Class member method
    public static void main(String[] args)
    {
        Car obj = new Car();
    }
}
```

- The instance initializer block is created when instance of the class is created
- The instance initializer block is invoked after the parent class constructor is invoked (i.e. after super() constructor call)
- The instance initializer block comes in the order in which they appear

- A lambda expression is an anonymous function that means a function that doesn't have a name and doesn't belong to any class
- In Java, Lambda expressions basically express instances of functional interfaces (An interface with a single abstract method is called a functional interface).
- Lambda Expressions in Java are the same as lambda functions which are the short block of code that accepts input as parameters and returns a resultant value.
- Lambda Expressions are recently included in Java SE 8.

Functionalities of Lambda Expression in Java

- Enable to treat functionality as a method argument, or code as data.
- A function that can be created without belonging to any class.
- A lambda expression can be passed around as if it was an object and executed on demand.

Java Lambda Expression Syntax

- To create a lambda expression, we specify input parameters (if there are any) on the left side of the lambda operator $\rightarrow$, and place the expression or block of statements on the right side of lambda operator
- For example, the lambda expression $(x, y) \rightarrow x + y$ specifies that lambda expression takes two arguments x and y and returns the sum of these
- //Syntax of lambda expression
- $(\text{parameter_list}) \rightarrow \{\text{function_body}\}$
- To use lambda expression, you need to either create your own functional interface or use the pre defined functional interface provided by Java
- An interface with **only single abstract method** is called functional interface(or Single Abstract method interface), for example: Runnable, callable, ActionListener etc

Java Lambda Expression - Example

```
// Java program to demonstrate lambda expressions to implement a user defined functional interface.
// A sample functional interface (An interface with single abstract method)
interface FuncInterface
{
    // An abstract function
    void abstractFun(int x);

    // A non-abstract (or default) function
    default void normalFun()
    {
        System.out.println("Hello");
    }
}

class Test
{
    public static void main(String args[])
    {
        // lambda expression to implement above functional interface. This interface by default implements abstractFun()
        FuncInterface fobj = (int x)->System.out.println(2*x);

        // This calls above lambda expression and prints 10.
        fobj.abstractFun(5);
    }
}
```

(int arg1, String arg2) -> {System.out.println("Two arguments "+arg1+" and "+arg2);}

Argument List Arrow token Body of lambda expression

Lambda Expression Parameters

There are three Lambda Expression Parameters are mentioned below:

- 1.Zero Parameter
- 2.Single Parameter
- 3.Multiple Parameters

1. Lambda Expression with Zero parameter

() -> System.out.println("Zero parameter lambda");

2. Lambda Expression with Single parameter

(p) -> System.out.println("One parameter: " + p);

3. Lambda Expression with Multiple parameters

(p1, p2) -> System.out.println("Multiple parameters: " + p1 + ", " + p2);

Lambda expression vs method in Java

- A method (or function) in Java has these main parts:
 1. Name
 2. Parameter list
 3. Body
 4. return type
- A lambda expression in Java has these main parts:

Lambda expression **only has body and parameter list**

 1. **No** name – function is anonymous so we don't care about the name
 2. Parameter list
 3. Body – This is the main part of the function
 4. **No** return type – The java 8 compiler is able to infer the return type by checking the code. you need not to mention it explicitly

